MINOR PROJECT REPORT

CNN Image Classification and Streamlit Application

Handwritten Digit Recognition using the MNIST Dataset

Student Name	Kavana M S
Project Type	Individual Minor Project
Selected Dataset	MNIST Handwritten Digits
Application	Streamlit Digit Recognition App
Submission Date	_____
Batch / Class	_____

Submitted as part of the Minor Project Assignment

1. Introduction

Image classification is a computer vision task in which a computer model analyses an image and assigns it to one of a set of predefined categories. This project uses a Convolutional Neural Network (CNN) to recognise handwritten digits. The selected MNIST dataset contains grayscale images of handwritten digits from 0 to 9.

1.1 Project Objectives

- Understand the image dimensions, class labels and number of samples in the dataset.
- Preprocess image data so that it can be used by a CNN.
- Build and train a CNN model for digit classification.
- Evaluate the trained model using test data and inspect sample predictions.
- Create a simple Streamlit application that accepts a digit image and displays a prediction.

1.2 Project Scope

The project has two parts. Part A covers dataset understanding, preprocessing, CNN model creation, training, testing and prediction examples. Part B provides a Streamlit user interface where a user can upload an image of a handwritten digit and view the predicted digit and confidence score.

2. Dataset Description

The project uses the MNIST handwritten digit dataset provided through the Keras dataset API. It contains 60,000 training images and 10,000 testing images. Each image is a 28 × 28 grayscale image and belongs to one of ten classes: 0, 1, 2, 3, 4, 5, 6, 7, 8 or 9.

Dataset property	Description
Training samples	60,000 images
Testing samples	10,000 images
Image dimensions	28 × 28 pixels
Image channels	1 (grayscale)
Number of classes	10 (digits 0–9)

Dataset source: Keras MNIST dataset loader (tf.keras.datasets.mnist). The dataset is downloaded automatically the first time the program runs with an internet connection.

3. CNN Methodology and Workflow

3.1 Data Loading and Preprocessing

The MNIST dataset is loaded into training and testing sets. Pixel values originally range from 0 to 255. They are converted to floating-point values and divided by 255 so that the input values are scaled between 0 and 1. A channel dimension is added to each image, changing the image shape from 28 × 28 to 28 × 28 × 1, which is the input format used by the CNN.

3.2 CNN Architecture

Layer	Purpose
Input: 28 × 28 × 1	Receives one grayscale digit image.
Convolution (32 filters, 3 × 3)	Learns basic image patterns and features.
Max Pooling (2 × 2)	Reduces feature-map size.
Convolution (64 filters, 3 × 3)	Learns more detailed patterns.
Max Pooling (2 × 2)	Further reduces spatial dimensions.
Flatten	Converts feature maps into a one-dimensional vector.
Dense (128 neurons, ReLU)	Learns relationships between extracted features.
Dropout (0.3)	Helps reduce overfitting during training.
Dense (10 neurons, Softmax)	Outputs probabilities for the ten digit classes.

3.3 Model Training

The model is compiled with the Adam optimiser, sparse categorical cross-entropy loss and accuracy as the evaluation metric. It is trained for 5 epochs with a batch size of 128. Ten percent of the training data is used for validation during training.

3.4 Model Evaluation

After training, the model is evaluated on the separate test dataset. Test accuracy and test loss are recorded. Sample test images are also passed to the model so that their actual labels can be compared with predicted labels.

4. Results and Observations

Complete this section after running the code in Google Colab or your local Python environment. Do not enter estimated values. Copy the actual values displayed by model.evaluate() and insert your own screenshots in the spaces below.

4.1 Model Performance

Performance measure	Actual result (fill after running)
Test accuracy	_____ %
Test loss	_____
Training accuracy (final epoch)	_____ %
Validation accuracy (final epoch)	_____ %

4.2 Training and Validation Accuracy

[Insert screenshot of the accuracy graph generated by the program here.]

4.3 Training and Validation Loss

[Insert screenshot of the loss graph generated by the program here.]

4.4 Sample Predictions

[Insert screenshot showing sample test images with their actual and predicted labels. Mention at least one correct prediction and, if available, one incorrect prediction.]

5. Streamlit Application

The Streamlit application provides a simple interface for using the trained CNN model. It displays dataset information, sample MNIST images, model summary, training graphs and test performance. A user can upload a PNG or JPG image containing one handwritten digit. The application preprocesses the uploaded image, sends it to the CNN and displays the predicted digit and its confidence score.

5.1 Main Application Features

- Clear application title and instructions.
- Dataset sample count, image size and digit classes.
- Training and validation accuracy/loss graphs.
- Test accuracy and test loss display.
- Image upload, digit prediction and class probability display.
- Basic error message for an image that cannot be processed.

5.2 Application Screenshot

[Insert a screenshot of the running Streamlit application showing its title, upload control and prediction output. Take this screenshot after successfully running the application.]

6. Conclusion

This project demonstrates how a Convolutional Neural Network can be used to classify handwritten digit images from the MNIST dataset. The work includes image preprocessing, CNN model construction, model training, test evaluation and sample prediction. The Streamlit application makes the model easier to use by allowing a user to upload a digit image and view the prediction. The final model performance should be reported using the actual test results obtained when the code is executed.

6.1 Challenges and Future Improvements

Possible challenges include differences in handwriting style, image brightness, image alignment and background noise. Future improvements could include better image centring, more robust preprocessing, data augmentation, hyperparameter tuning and testing with a wider variety of handwritten images.

Note: This report is a ready-to-complete draft. Replace all bracketed screenshot instructions and blank result fields with your actual outputs before submitting.